

Experiment 29: Backward Chaining Using Prolog

Aim

To implement **Backward Chaining using Prolog** and demonstrate how Prolog starts with a goal and works backward through rules and facts to prove the goal.

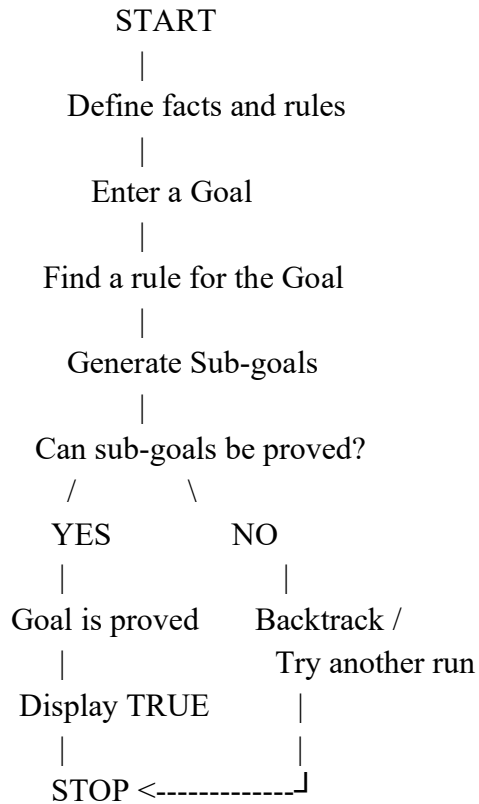
Objective

1. To understand the concept of backward chaining.
2. To represent knowledge using Prolog facts and rules.
3. To prove a goal by working backward from the query.
4. To execute queries and observe the logical inference performed by Prolog.

Algorithm

1. Start the program.
2. Define facts about birds and their properties.
3. Define a rule to identify an animal from a bird.
4. Define a rule to identify a flying animal.
5. Enter a goal/query.
6. Prolog searches for a rule that can prove the goal.
7. Prolog generates sub-goals from the selected rule.
8. Prolog checks whether the sub-goals can be proved using available facts or other rules.
9. If all sub-goals are satisfied, the original goal is proved.
10. Display the result.
11. Stop.

Flowchart



Prolog Code

% Backward Chaining Example

% Facts

bird(sparrow).

bird(eagle).

has_wings(sparrow).

has_wings(eagle).

can_fly(sparrow).

can_fly(eagle).

% Rules

animal(X) :-

bird(X).

```
flying_animal(X) :-  
    animal(X),  
    has_wings(X),  
    can_fly(X).
```

Sample Output

Query 1: Is Sparrow a bird?

```
?- bird(sparrow).
```

Query 2: Is Sparrow an animal?

```
?- animal(sparrow).
```

Query 3: Can Sparrow fly?

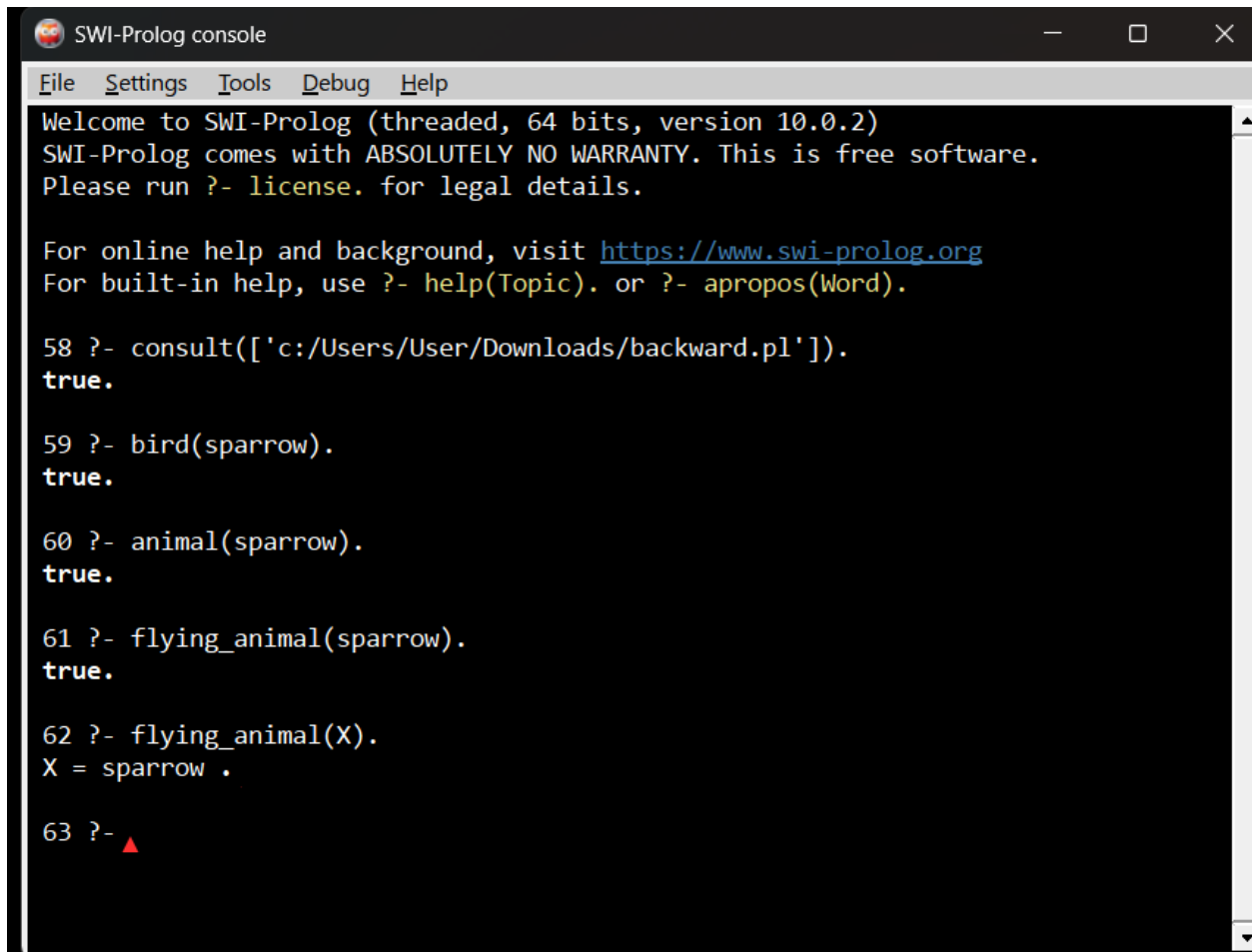
```
?- flying_animal(sparrow).
```

Query 4: Find all flying animals

```
?- flying_animal(X).
```

Query 5: Is Dog a flying animal?

```
?- flying_animal(dog).
```

A screenshot of the SWI-Prolog console window. The window has a title bar with the SWI-Prolog logo and the text "SWI-Prolog console". Below the title bar is a menu bar with "File", "Settings", "Tools", "Debug", and "Help". The main area of the window is a black terminal with white text. It shows the welcome message for SWI-Prolog version 10.0.2, followed by instructions to run "?- license." for legal details and to visit <https://www.swi-prolog.org> for online help. The user has entered several queries: "?- consult(['c:/Users/User/Downloads/backward.pl'])." which returns "true.", "?- bird(sparrow)." which returns "true.", "?- animal(sparrow)." which returns "true.", "?- flying_animal(sparrow)." which returns "true.", and "?- flying_animal(X)." which returns "X = sparrow .". The prompt "?- " is followed by a red triangle cursor.

```
SWI-Prolog console
File Settings Tools Debug Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.2)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

58 ?- consult(['c:/Users/User/Downloads/backward.pl']).
true.

59 ?- bird(sparrow).
true.

60 ?- animal(sparrow).
true.

61 ?- flying_animal(sparrow).
true.

62 ?- flying_animal(X).
X = sparrow .

63 ?- ▲
```

Result

The **Backward Chaining program** was **successfully implemented using Prolog**. The program successfully proved the given goals by working backward from the query through rules and facts.

Conclusion

Thus, the experiment demonstrates the use of **backward chaining in Prolog**. Prolog begins with a goal, identifies the rules that can satisfy the goal, creates sub-goals, and checks the available facts to determine whether the original goal can be proved.